

Dónde está cada requisito

Los 26 puntos de la lista de chequeo, en su orden, con la pantalla donde se demuestra cada uno, el archivo donde vive y su captura. Pensado para tenerlo abierto al lado mientras se sustenta.

Ninguna captura está preparada a mano: las tomó un navegador dirigido por herramientas/capturas.py sobre el proyecto en marcha, y las de consola son la salida literal de cada comprobación. Si mañana algo deja de funcionar, se vuelve a lanzar y la imagen lo enseña. Esa es la diferencia entre una evidencia y una ilustración.

Antes de empezar

Ninguna de las direcciones de abajo responde si falta una de estas tres piezas. Se levantan en este orden.

MySQL:3306 Botón <i>Start</i> de MySQL en el panel de XAMPP.	API FastAPI:8000 Desde la carpeta backend: venv\Scripts\python -m uvicorn app.main:app --reload --port 8000	Interfaz Vite:5173 Desde la carpeta frontend: npm run dev
--	---	---

Cuentas para la demostración

Las tres existen desde el tercer avance y siguen sirviendo: bcrypt es el mismo algoritmo y el mismo formato, así que el cambio de Node a Python no obligó a nadie a registrarse de nuevo.

Cambiar de cuenta es la forma más rápida de enseñar los tres paneles y los 403.		
CORREO	CONTRASEÑA	ROL
admin@autoprime.com.co	Admin2026!	administrador
empleado@autoprime.com.co	Empleado2026!	empleado
cliente@autoprime.com.co	Cliente2026!	cliente

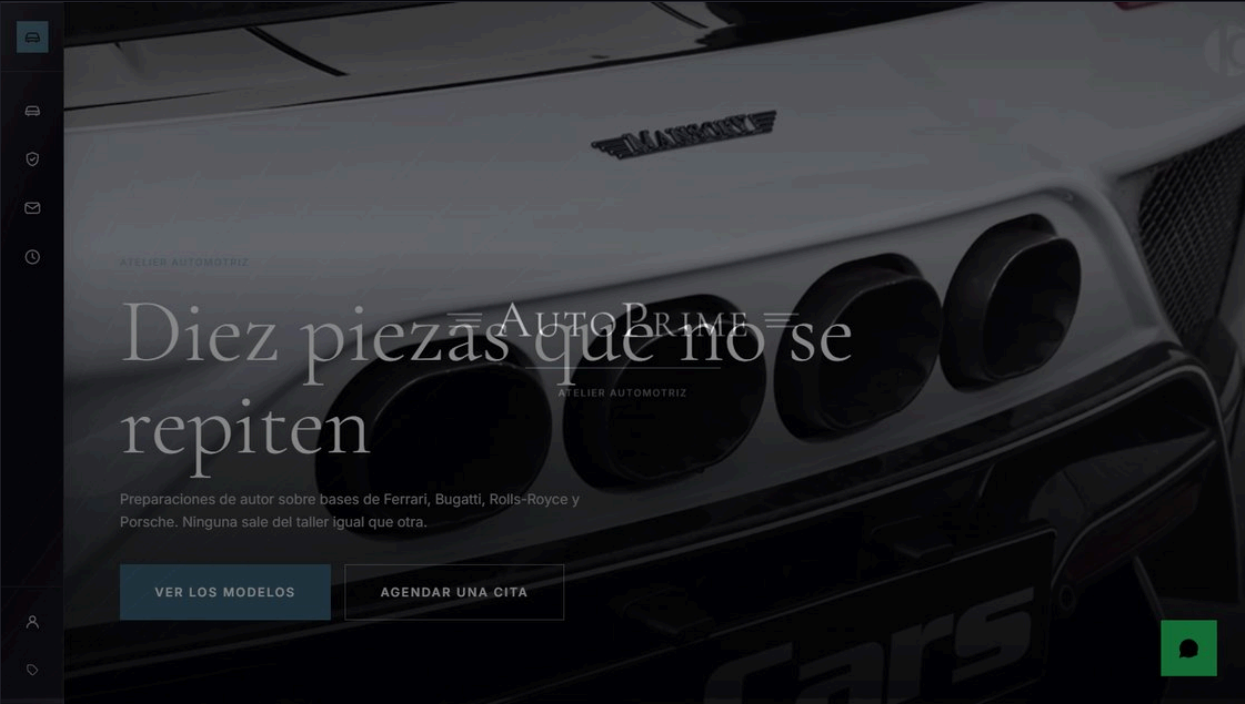
Los 26 requisitos

En el orden del instrumento, para poder ir tachando. Las rutas de archivo son relativas a la raíz del proyecto; los números son de línea.

Arquitectura tecnológica

React + Vite consumiendo endpoints de FastAPI en JSON, y el backend conectado a una base SQL relacional.

SE VE EN	ESTÁ EN
Cualquier vista con datos, por ejemplo /agendar. Con las herramientas del navegador en la pestaña <i>Red</i> se ven las llamadas al puerto 8000 devolviendo JSON.	<code>frontend/src/api/cliente.js:10</code> – dirección base <code>frontend/src/api/cliente.js:64</code> – la llamada <code>fetch</code> <code>backend/app/core/base_datos.py:10</code> – motor <code>SQLAlchemy</code>



EVIDENCIA La portada servida por Vite, consumiendo la API.

Carpeta backend con FastAPI

Frontend y backend separados, con el backend organizado en una subcarpeta app/ y sus dependencias.

SE VE EN	ESTÁ EN
No tiene pantalla: se enseña el árbol de carpetas del proyecto.	backend/app/ – core, models, schemas, crud, routers
	backend-express/ – el backend anterior, como evidencia

Estructura del proyectoREQ-02

Frontend y backend separados; el backend con su subcarpeta app/.

```
$ tree proyecto-final
├── Evidencias_Tercer_Avance_AutoPrime.pdf
├── README.md
├── backend/
├── backend-express/
├── design-system/
├── evidencias/
├── frontend/
├── herramientas/
├── package-lock.json
└── package.json

$ tree backend/app
├── __init__.py
├── core/
├── crud/
├── dependencias.py
├── errores.py
├── main.py
├── models/
├── routers/
└── schemas/
```

EVIDENCIA Árbol del proyecto y del paquete `app`.

Entorno FastAPI

Entorno virtual de Python activado y dependencias principales registradas en requirements.txt.

SE VE EN	ESTÁ EN
La terminal al arrancar la API: aparece el arranque de uvicorn y <i>Application startup complete</i> .	backend/requirements.txt:4 – fastapi 0.141.1
	backend/requirements.txt:11 – SQLAlchemy 2.0.52
	backend/venv/ – el entorno

Entorno virtual y dependenciasREQ-03

Dependencias principales instaladas en el entorno del proyecto.

```
$ venv\Scripts\python --version
Python 3.14.4

$ venv\Scripts\python -m pip list
bcrypt 5.0.0
fastapi 0.141.1
pydantic 2.13.5
pydantic_core 2.46.5
pydantic-settings 2.15.0
PyMySQL 1.2.0
python-jose 3.5.0
SQLAlchemy 2.0.52
uvicorn 0.52.4
```

EVIDENCIA Versión de Python y dependencias del entorno.

Base de datos SQL

Estructura relacional con, como mínimo, usuarios, roles, permisos, productos y servicios.

SE VE EN	ESTÁ EN
phpMyAdmin, base autoprima: siete tablas, incluida rol_permiso que enlaza las dos primeras.	backend/sql/autoprima.sql:18 – roles backend/sql/autoprima.sql:27 – permisos backend/sql/autoprima.sql:33 – rol_permiso backend/app/models/autoprima.py – el mapeo

Tablas de la base de datos

REQ-04

Las siete tablas del esquema relacional, incluidas roles y permisos.

```
$ mysql -u root -e "SHOW TABLES FROM autoprima;"
+-----+
| Tables_in_autoprima |
+-----+
| citas                |
| permisos             |
| productos            |
| rol_permiso          |
| roles                |
| servicios            |
| usuarios             |
+-----+
```

EVIDENCIA Las siete tablas del esquema.

Tabla de usuarios

Nombre, apellido, tipo y número de documento, dirección, teléfono, correo, contraseña, rol y estado. La contraseña, solo con hash seguro.

SE VE EN	ESTÁ EN
phpMyAdmin, tabla usuarios: la columna se llama password_hash y guarda cadenas \$2b\$10\$,, nunca la contraseña.	backend/sql/autoprima.sql:46 – las 13 columnas backend/app/models/autoprima.py:53 – clase Usuario

Estructura de la tabla usuarios

REQ-05

Todos los campos que pide el entregable. La contraseña solo como hash.

```
$ mysql -u root -e "DESCRIBE autoprima.usuarios;"
+-----+-----+-----+-----+-----+-----+
| Field      | Type          | Null | Key | Default | Extra           |
+-----+-----+-----+-----+-----+-----+
| id         | int(11)       | NO   | PRI | NULL    | auto_increment |
| nombre     | varchar(40)   | NO   |     | NULL    |                 |
| apellido   | varchar(40)   | NO   |     | NULL    |                 |
| tipo_documento | enum('CC','TI','CE','PA','NIT') | NO   | MUL | NULL    |                 |
| numero_documento | varchar(15)   | NO   |     | NULL    |                 |
| direccion  | varchar(80)   | NO   |     | NULL    |                 |
| telefono   | varchar(10)   | NO   |     | NULL    |                 |
| correo     | varchar(60)   | NO   | UNI | NULL    |                 |
| password_hash | varchar(255)  | NO   |     | NULL    |                 |
| rol_id     | int(11)       | NO   | MUL | NULL    |                 |
| estado     | enum('activo','inactivo') | NO   | MUL | activo  |                 |
| creado_en  | timestamp     | NO   |     | current_timestamp() |                 |
| actualizado_en | timestamp     | NO   |     | current_timestamp() | on update current_timestamp() |
+-----+-----+-----+-----+-----+-----+
```

EVIDENCIA Campos de `usuarios`; la contraseña es `password\_hash`.

Modelos y esquemas

SQLAlchemy para la base y Pydantic para validar, separados. Los esquemas validan tipos, obligatorios, longitudes y formatos.

SE VE EN	ESTÁ EN
El apartado <i>Schemas</i> , al final de localhost:8000/docs. Un usuario tiene password_hash en el modelo y ese campo no aparece en ningún esquema de salida.	backend/app/models/autoprima.py – filas de la base backend/app/schemas/ – lo que entra y sale



EVIDENCIA Los esquemas de Pydantic publicados en la documentación.

Conexión de la base con FastAPI

Conexión funcional con SQL y parámetros por variables de entorno, sin contraseñas en el código fuente.

SE VE EN	ESTÁ EN
localhost:8000/salud responde "base_datos": "conectada". Es una consulta real, no un valor fijo.	backend/app/core/base_datos.py:10 – el motor backend/app/core/configuracion.py:53 – cadena de conexión backend/.env.example – la plantilla

Dar formato al texto

```
{"ok":true,"base_datos":"conectada","entorno":"desarrollo"}
```

EVIDENCIA La API responde tras consultar la base de verdad.

Conexión frontend, backend y base

El formulario de React envía a FastAPI, que revalida por su cuenta, con independencia del frontend, y opera sobre la base SQL.

SE VE EN	ESTÁ EN
Registrarse con un correo ya existente: el aviso llega del backend con un 409, no del navegador. Se comprueba mejor desde Swagger, saltándose la interfaz por completo.	backend/app/crud/usuarios.py:64 – comprobar duplicados backend/app/schemas/usuario.py – reglas de entrada

Le API revalida por su cuenta

REQ-08

Petición hecha sin pasar por el formulario de React. La regla de correo único se aplica igual.

\$ POST /api/auth/registro

{ "nombre": "Prueba", "apellido": "Duplicada", "tipoDocumento": "CC", "numeroDocumento": "1088111222", "direccion": "Calle 1 # 2-3", "telefono": "3001112233", "correo": "admin@autoprime.com.co" }

HTTP 409

{ "codigo": "correo_ya_registrado", "mensaje": "Ya existe una cuenta con el correo admin@autoprime.com.co.", "ruta": "/api/auth/registro", "detalles": null }

EVIDENCIA Alta duplicada pedida sin pasar por el formulario.

Registro de clientes

Formulario conectado a POST /api/usuarios/registro: validaciones, verificación de no duplicados, hash de la contraseña, guardado y respuesta JSON.

SE VE EN	ESTÁ EN
/login → botón <i>Crear cuenta</i> . La respuesta deja la sesión ya iniciada.	backend/app/routers/usuarios.py:28 – la ruta de la lista
	backend/app/routers/auth.py:37 – la misma función
	frontend/src/components/auth/RegisterModal.jsx

Crear una cuenta

Completar tus datos para acceder a tu perfil de cliente.

NOMBRE

Jose Matias

APELLIDO

Agudelo Bolívar

TIPO DE DOCUMENTO

Selecciona una opción

NÚMERO DE DOCUMENTO

1012345678

Solo números, entre 6 y 11 dígitos.

DIRECCIÓN

Calle 45 # 12-30, Barrio Centro

0/80

TELÉFONO

3001234567

Celular colombiano de 10 dígitos.

CORREO ELECTRÓNICO

nombre@correo.com

CONTRASEÑA

Mínimo 8 caracteres

CONFIRMAR CONTRASEÑA

Repite la contraseña

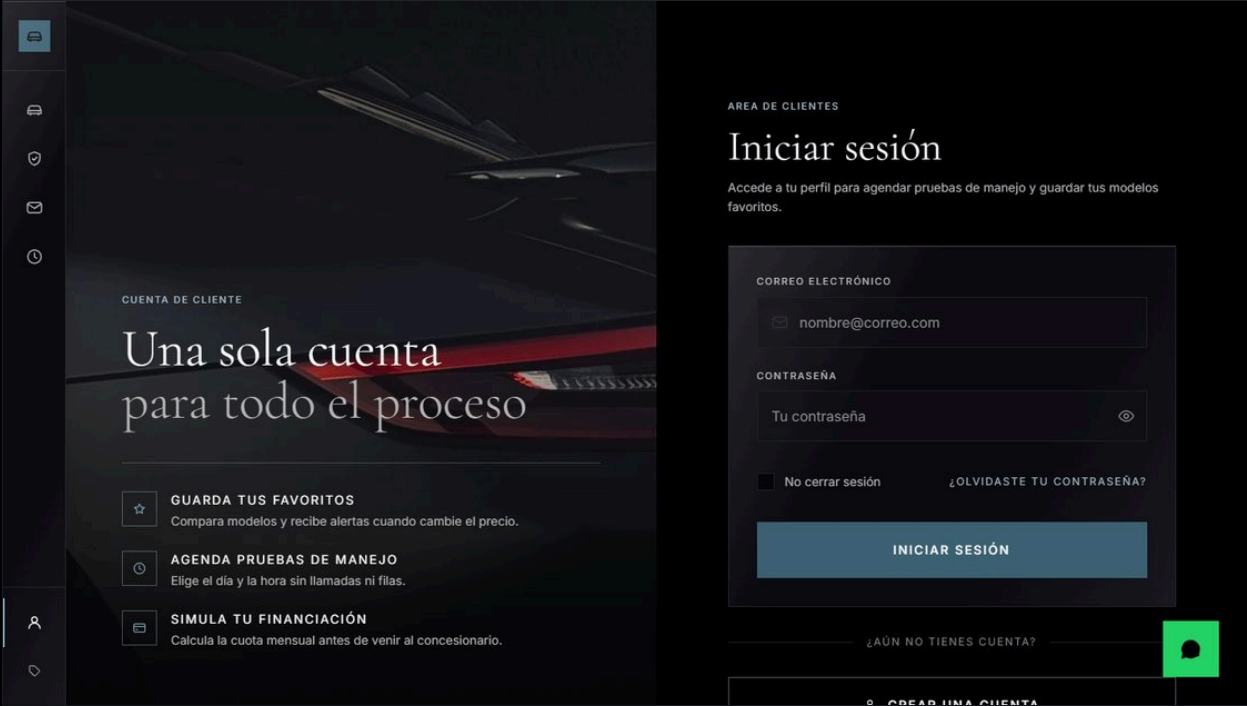
☐ Acepto los términos y el tratamiento de mis datos personales.

EVIDENCIA Formulario de alta de cliente.

Inicio de sesión

Formulario conectado a POST /api/auth/login. FastAPI verifica las credenciales y genera un JWT si son correctas.

SE VE EN	ESTÁ EN
/login. Cada rol aterriza en su propio panel.	backend/app/routers/auth.py:54 frontend/src/context/AuthContext.jsx:47



EVIDENCIA Formulario de acceso.

Autenticación con JWT

React envía Authorization: Bearer en las peticiones que lo requieren; FastAPI verifica firma, validez, expiración, usuario y rol.

SE VE EN	ESTÁ EN
Herramientas del navegador → <i>Red</i> → cualquier petición del panel: la cabecera Authorization viaja en todas.	backend/app/core/seguridad.py:42 – emitir backend/app/dependencias.py:32 – verificar frontend/src/api/cliente.js:59 – enviar

■ ■ ■ Contenido del JSON Web Token

REQ-11

El token que emite FastAPI al iniciar sesión, descodificado. Lleva el usuario, su rol y la caducidad; la firma es lo que impide alterarlo.

\$ token emitido por POST /api/auth/login

cabecera:

{
 "alg": "HS256",
 "typ": "JWT"
}

carga:

{
 "sub": "1",
 "tipo": "sesion",
 "correo": "admin@autoprima.com.co",
 "rol": "administrador",
 "iat": 1788549289,
 "exp": 1788578889
}

firma: 4R_otQxiPDf6BFkXeULE8hrp83T6ZpVCaTq2SceT... (no descodificable: es la prueba)

EVIDENCIA El token descodificado: usuario, rol y caducidad.

Control de roles

Administrador, empleado y cliente. Control en frontend y backend, con la autorización definitiva en el backend.

SE VE EN	ESTÁ EN
Entrando como cliente e intentando abrir /panel/admin: la interfaz desvía. La prueba de fondo es repetirlo en Postman con el token del cliente, que responde 403 sin interfaz de por medio.	backend/app/dependencias.py:73 – permitir_rol backend/app/dependencias.py:91 – SoloAdmin y Personal frontend/src/components/RutaProtegida.jsx:28

■ ■ ■ El backend decide el permiso

REQ-12

El mismo endpoint, con el token de un cliente. La interfaz ya lo oculta, pero la autorización de verdad ocurre aquí.

\$ GET /api/usuarios

Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6I...

HTTP 403

{
 "codigo": "permiso_denegado",
 "mensaje": "Esta operación está reservada al rol administrador.",
 "ruta": "/api/usuarios",
 "detalles": null
}

EVIDENCIA Un cliente pidiendo la lista de usuarios.

Uso de archivos y hooks

Hooks para gestionar los distintos estados de la aplicación: `useState`, `useEffect`, `useContext` y demás.

SE VE EN	ESTÁ EN
<i>No tiene pantalla propia: sostiene todas. La sesión que persiste al recargar es el ejemplo más visible.</i>	<code>frontend/src/hooks/</code> – siete hooks propios <code>frontend/src/context/AuthContext.jsx:15</code> – <code>createContext</code> <code>frontend/src/hooks/useAuth.js:6</code> – <code>useContext</code>

Hooks de ReactREQ-13

Siete hooks propios más los de la biblioteca, repartidos por la aplicación.

```
$ ls frontend/src/hooks
useAuth.js
useBloqueoScroll.js
useCarga.js
useFormulario.js
useIdioma.js
useReflejoPuntero.js
useScroll.js

$ archivos que usan cada hook de React
useState      22 archivos
useEffect     17 archivos
useContext     2 archivos
useRef         7 archivos
useMemo        4 archivos
useCallback   12 archivos
```

EVIDENCIA Hooks propios y uso de los de React.

Endpoints de la API

Endpoints para gestionar usuarios, productos y servicios, con rutas REST estructuradas por entidad.

SE VE EN	ESTÁ EN
localhost:8000/docs: 32 endpoints agrupados en cinco secciones, una por recurso.	backend/app/routers/ – un archivo por recurso backend/app/main.py:170 – se montan aquí

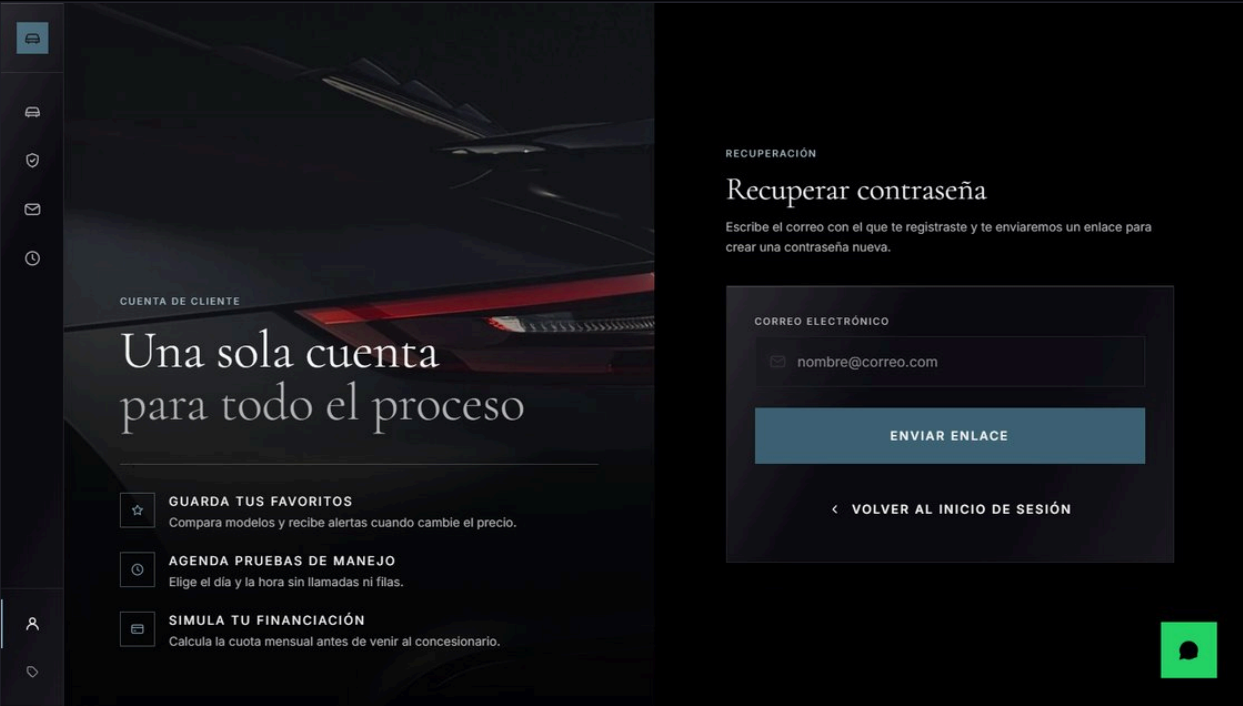


EVIDENCIA Catálogo servido por 'GET /api/productos'.

Recuperación de contraseña

Funcionalidad de recuperación de la contraseña olvidada por el usuario.

SE VE EN	ESTÁ EN
/login → ¿Olvidaste tu contraseña? El enlace llega al correo y se abre en /restablecer. El token no vuelve en la respuesta: ese rodeo por el buzón es lo que obliga a demostrar que la cuenta es tuya. Caduca a los 30 minutos, sirve una sola vez y no vale como token de sesión.	backend/app/routers/auth.py:97 – pedir y enviar backend/app/routers/auth.py:139 – canjearlo backend/app/core/correo.py:138 – el envío SMTP frontend/src/pages/RestablecerPassword.jsx



EVIDENCIA Primer paso de la recuperación.

Operaciones CRUD de usuarios

Consultar, crear, editar, eliminar y cambiar estado —activo o inactivo— para mantener la consistencia histórica.

SE VE EN	ESTÁ EN
/panel/admin, apartado de usuarios. Inactivar una cuenta y tratar de entrar con ella deja claro para qué sirve el estado.	backend/app/routers/usuarios.py:61 – crear backend/app/routers/usuarios.py:89 – cambiar estado frontend/src/pages/panel/PanelAdmin.jsx:97



EVIDENCIA Alta desde el panel, con selección de rol.

Panel de administración







PANEL DE ADMINISTRACIÓN

Gestión de usuarios

Consulta, agrega, edita, activa o elimina las cuentas del sistema. Solo el rol administrador tiene acceso a esta pantalla.

SESIÓN
Jose Matias Agudelo Bolivar
ADMINISTRADOR

 AGREGAR USUARIO

USUARIOS 16	ADMINISTRADORES 1	EMPLEADOS 1	CLIENTES 14
CITAS PENDIENTES	CONFIRMADAS	COMPLETADAS	CANCELADAS

FILTRAR POR ROL

Todos los roles

FILTRAR POR ESTADO

Todos los estados

BUSCAR

 Nombre, correo o documento

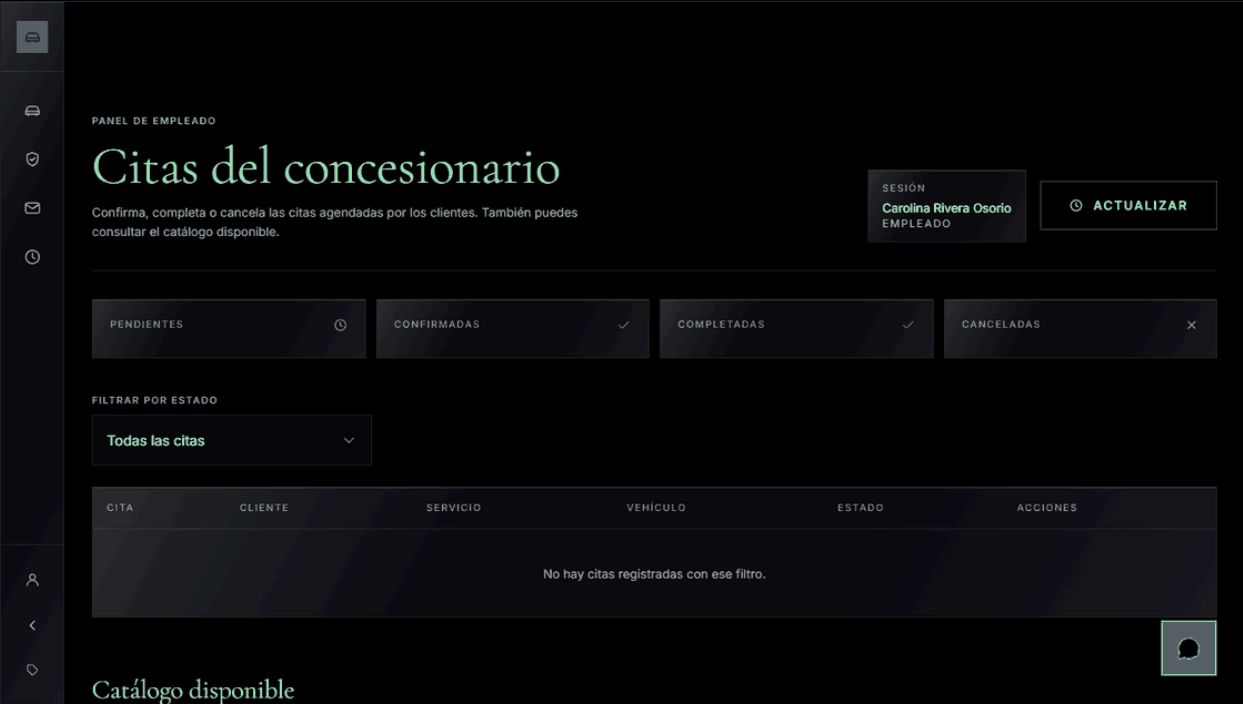
USUARIO	DOCUMENTO	CONTACTO	ROL	ESTADO	ACCIONES
Jose Matias Agudelo Bolivar (TÚ) admin@autoprime.com.co	CC 1088111222	300112233 Av. Las Americas 45-12	ADMINISTRADOR	ACTIVO	EDITAR INACTIVAR ELIMINAR

EVIDENCIA Panel de administración.

Panel de empleado

Muestra únicamente las funcionalidades del rol de empleado; el backend valida el rol antes de permitir operaciones restringidas.

SE VE EN	ESTÁ EN
/panel/empleado. No aparece la gestión de usuarios, que es exclusiva del administrador.	frontend/src/pages/panel/PanelEmpleado.jsx frontend/src/router/AppRouter.jsx:86

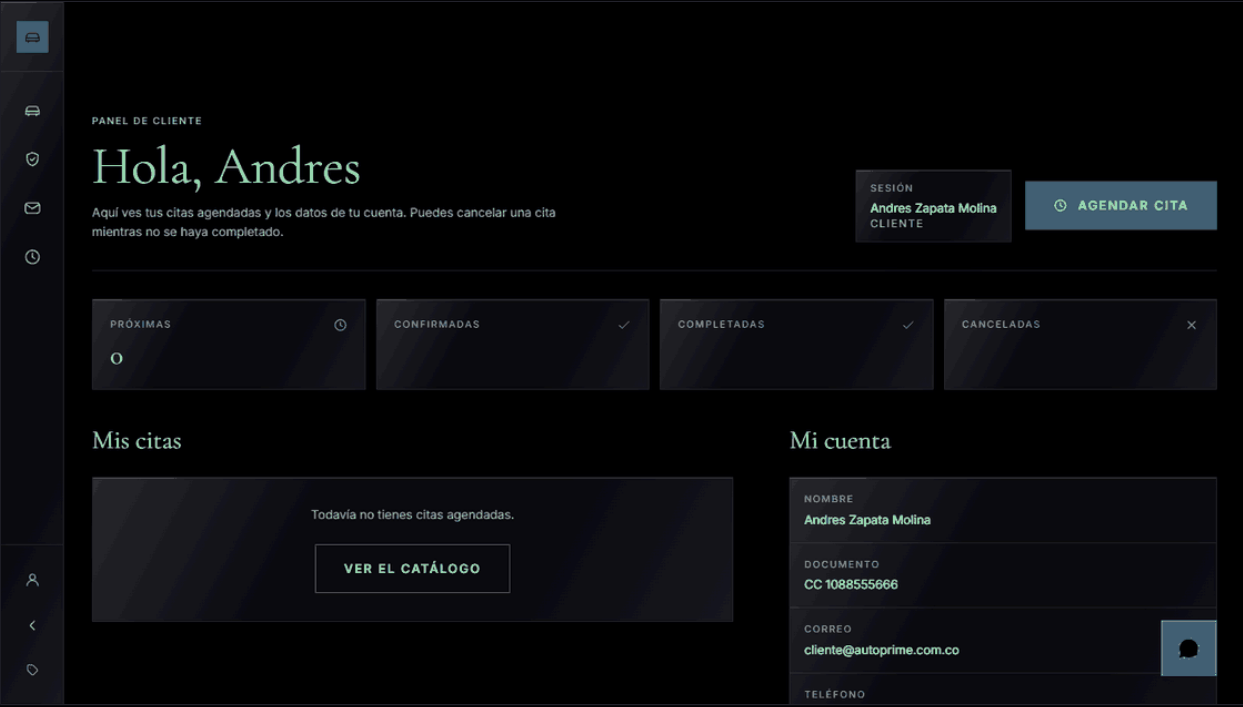


EVIDENCIA Panel de empleado.

Panel de cliente

El cliente accede a sus servicios y productos. El sistema lo identifica con la información del JWT.

SE VE EN	ESTÁ EN
/panel/cliente: solo salen sus citas. El dueño se saca del token, no de lo que envíe el navegador.	frontend/src/pages/panel/PanelCliente.jsx backend/app/routers/citas.py – filtra por dueño

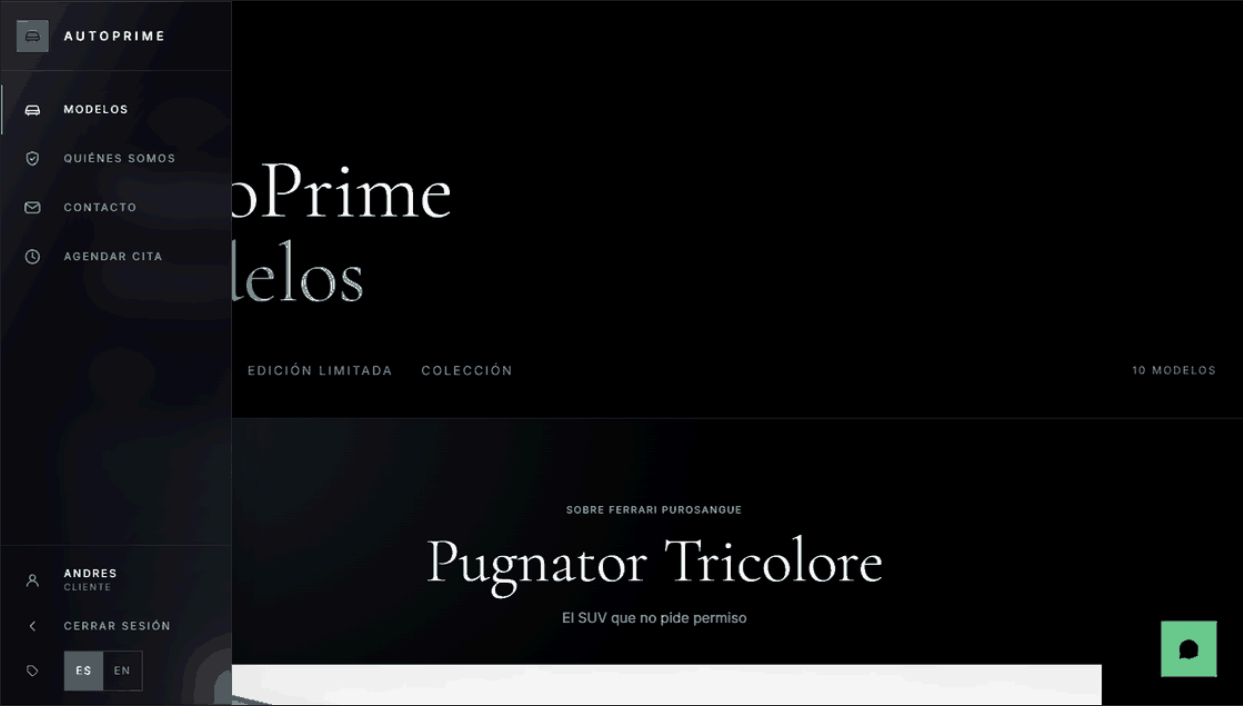


EVIDENCIA Panel de cliente.

Usuario en la barra de navegación

Tras iniciar sesión, React muestra el nombre del usuario autenticado, y se actualiza al cerrar sesión.

SE VE EN	ESTÁ EN
Arriba a la derecha, en cualquier página: <i>Bienvenido, «nombre»</i> . Al cerrar sesión vuelve a mostrar el acceso.	frontend/src/components/Header.jsx:142

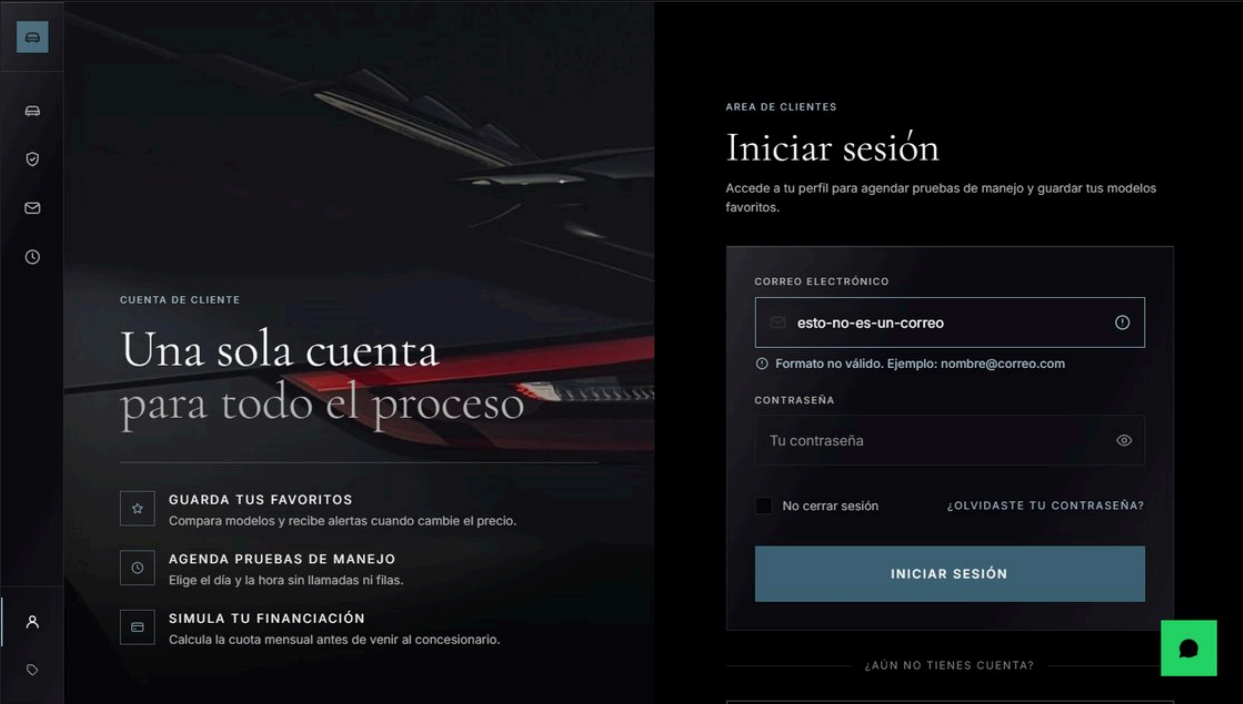


EVIDENCIA El nombre y el rol, en la navegación.

Validaciones en tiempo real

Campos obligatorios, longitudes, tipos, expresiones regulares, correos, teléfonos y contraseñas. Obligatorias tanto en React como en FastAPI.

SE VE EN	ESTÁ EN
El registro: un teléfono con letras avisa según se escribe, sin llegar a enviarse. La misma regla vuelve a aplicarse en la API, porque no puede fiarse de que la petición venga de ese formulario.	frontend/src/utils/validaciones.js – 12 validadores backend/app/schemas/comunes.py:68 – las mismas reglas



EVIDENCIA Aviso mientras se escribe, sin llegar a enviar.

Seguridad de las contraseñas

Prohibido el texto plano en la base. bcrypt para generar, almacenar y verificar en el inicio de sesión.

SE VE EN	ESTÁ EN
phpMyAdmin, columna password_hash. Dos cuentas con la misma contraseña tienen hashes distintos, porque cada una lleva su propia sal.	backend/app/core/seguridad.py:19 – hashear backend/app/core/seguridad.py:25 – verificar



EVIDENCIA Hashes bcrypt: misma longitud, valores distintos.

Variables de entorno

Un archivo .env local para resguardar información sensible: contraseñas de la base, claves secretas y credenciales.

SE VE EN	ESTÁ EN
backend/.env en el disco, y su ausencia en GitHub:	backend/.env.example – la plantilla
lo que se publica es la plantilla con valores de ejemplo.	backend/app/core/configuracion.py – cómo se leen
	backend/.gitignore:4 – por qué no se sube

Variables de entornoREQ-23

La plantilla se publica; el .env real con las claves queda fuera del repositorio.

```
$ type backend/.env.example
# Plantilla de configuracion del backend.
# Copiar a .env y rellenar: cp .env.example .env

ENTORNO=desarrollo

# Base de datos MySQL / MariaDB de XAMPP
DB_HOST=localhost
DB_PORT=3306
DB_USER=root
DB_PASSWORD=
DB_NAME=autoprime

# Clave de firma de los JWT
SECRET_KEY=cambiar-por-una-cadena-larga-y-aleatoria
ALGORITMO_JWT=HS256
HORAS_EXPIRACION_TOKEN=8
# Vida del enlace de recuperacion de contraseña
MINUTOS_EXPIRACION_RECUPERACION=30
BCRYPT_ROUNDS=10

# --- Correo saliente ---
# Direccion publica del frontend: con ella se arma el enlace del correo.
URL_FRONTEND=http://localhost:5173

# Servidor de correo para enviar el enlace de recuperacion.
# Si se deja SMTP_HOST vacio, el enlace se escribe en el registro del servidor
# en lugar de enviarse, y el flujo se puede probar igual.
#
# Con Gmail hace falta una CONTRASEÑA DE APLICACION (no la del correo):
# 1. Activa la verificacion en dos pasos en la cuenta de Google.
# 2. Entra en https://myaccount.google.com/apppasswords
# 3. Genera una y pegala en SMTP_PASSWORD, sin espacios.
SMTP_HOST=
SMTP_PUERTO=587
SMTP_USUARIO=
SMTP_PASSWORD=
# Opcional; por defecto se usa SMTP_USUARIO.
SMTP_REMITENTE=

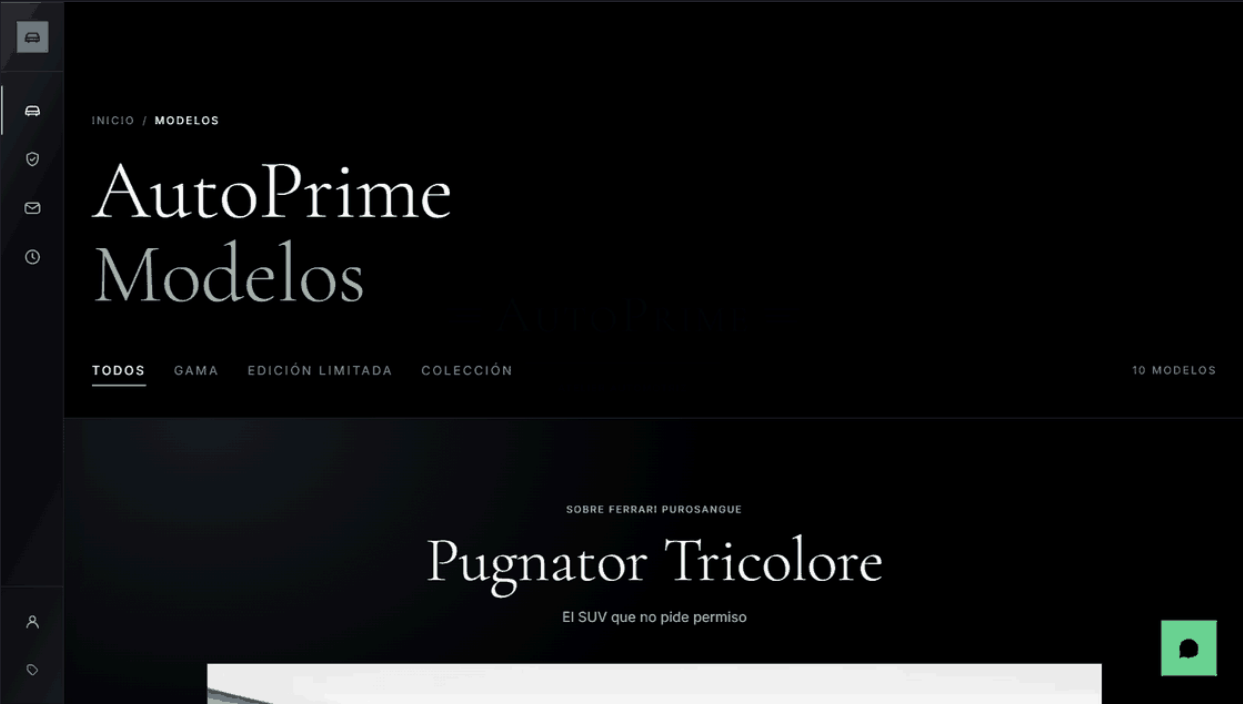
$ git check-ignore -v backend/.env
backend/.gitignore:4:.env      backend/.env
```

EVIDENCIA Plantilla publicada y .env real ignorado.

Componente flotante de WhatsApp

Botón reutilizable WhatsAppButton.jsx con posición fija, enlace configurado y diseño coherente. Funciona con independencia del backend.

SE VE EN	ESTÁ EN
Abajo a la derecha, en todas las páginas. Sigue ahí con la API apagada: no depende de ella.	frontend/src/components/WhatsAppButton.jsx:23 – posición fija frontend/src/router/AppRouter.jsx:110 – fuera de las rutas



EVIDENCIA El botón flotante, abajo a la derecha.

Documentación de FastAPI

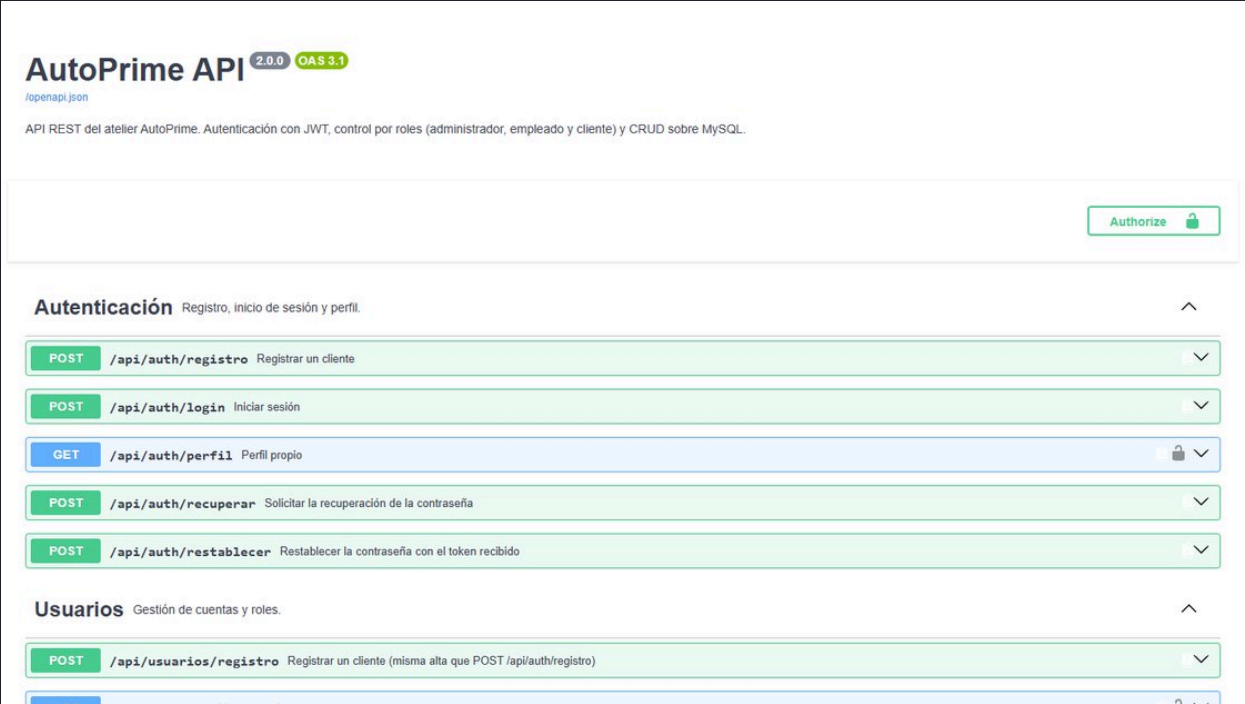
Documentación interactiva habilitada y visible; Swagger UI como evidencia del funcionamiento de la API.

SE VE EN

localhost:8000/docs. El botón *Authorize* admite el token del login y permite probar allí mismo lo protegido.

ESTÁ EN

backend/app/main.py:53 – se habilita aquí

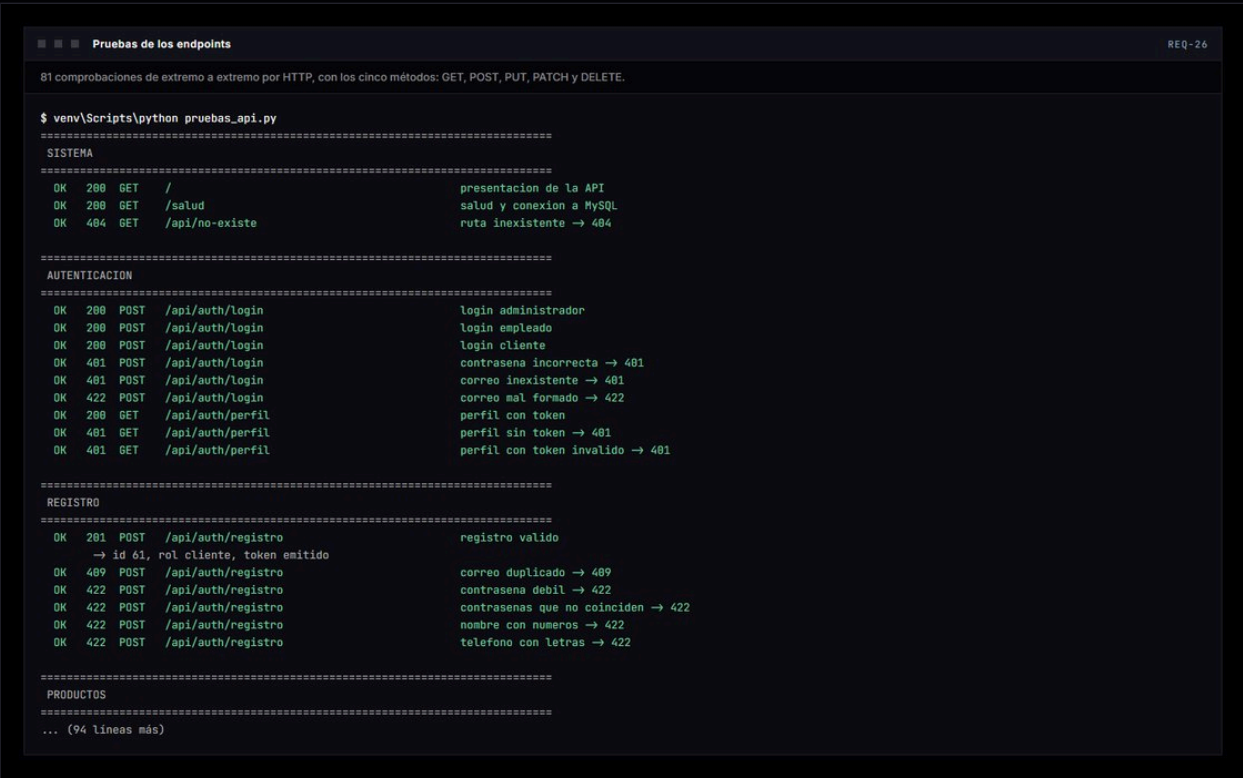


EVIDENCIA Documentación interactiva en `/docs`.

Pruebas con Postman

Evidencias de pruebas de los endpoints, demostrando los métodos GET, POST, PUT, PATCH y DELETE.

SE VE EN	ESTÁ EN
Postman, importando la colección: 44 peticiones en 6 carpetas, con los cinco métodos. El primer login guarda el token solo, así que el resto funciona sin copiarlo a mano.	backend/sql/AutoPrime.postman_collection.json backend/pruebas_api.py – 81 pruebas automáticas



EVIDENCIA Las 81 comprobaciones por HTTP.

Para rellenar la lista de chequeo

Las 26 imágenes están sueltas en evidencias/capturas/, con el número de requisito en el nombre, para arrastrarlas a los recuadros del instrumento sin tener que ir buscando cuál es cuál.

Queda una casilla que solo puede rellenar el aprendiz: la fila final con la dirección donde está alojado el entregable. Es github.com/matias2421/autoprima.

Para rehacerlas después de cualquier cambio, con MySQL, la API y Vite en marcha:
python herramientas/capturas.py, evidencias_consola.py, recortar.py y
montar_evidencias.py, en ese orden.